Modalités et consignes

FAQ

Tout au long du projet, nous publierons des réponses à des questions ou des précisions pour le projet. N'oubliez pas d'en vérifier régulièrement le contenu (voir Moodle).

Contraintes d'implémentation

Voici quelques contraintes d'implémentation dont vous devez tenir compte :

- Tous les champs des formulaires seront validés à l'aide de contraintes conformes à l'énoncé et choisies avec bon sens. Ces formulaires seront pré-remplis avec les données précédemment soumises en cas d'erreur de validation.
- Vous respecterez les bonnes pratiques de programmation web (HTTP, HTML, CSS) et veillerez à ce que toutes vos pages soient conformes au standard W3C HTML5. Vérifiez les à l'aide du validateur W3C ou, pour gagner du temps, à l'aide d'une extension de votre navigateur comme HTML Validator pour Chrome.
- Vous respecterez et baserez vos développements sur le **framework MVC** vu au cours (sans modifier le code fourni).
- Vous veillerez à typer au maximum les méthodes de vos classes.
- Les constantes de longueur minimum ou maximum des règles métier doivent être définies dans le fichier de configuration.
- Le projet doit pouvoir être exécuté sur une machine de l'école dans l'environnement `Xampp` et votre code doit être compatible avec la version 8.2.12 de PHP.
- Vous respecterez systématiquement le patron de conception **Post-Request-Get** vu au cours.
- Vous rendrez toutes les URL's de vos requêtes GET **univoques** (et donc *bookmarkables*) : cela signifie que l'URL (y compris ses paramètres) induit, de manière déterministe, la page et les données qui doivent y être affichées. Par exemple, l'URL `http://localhost/prwb_2526_xyy/item/delete_confirm/123` indique de manière univoque que la page à afficher est celle de la confirmation de suppression d'une annonce dont l'identifiant est `123` (pour autant qu'on soit connecté, que l'annonce `123` existe et que l'utilisateur connecté ait le droit de le supprimer).
- Pour cette itération, vous ne pouvez pas utiliser de script client (JavaScript). Si vous utilisez Bootstrap ou une librairie CSS équivalente, vous ne pouvez pas en utiliser les

composants qui font appel à JavaScript, par exemple les boîtes de dialogue modales de Bootstrap. En cas de doute, posez la question à votre professeur.

Modalités pratiques et consignes

Le projet sera réalisé par **groupe de deux ou trois étudiants** conformément au **framework et aux technologies vues au cours** (PHP, MySQL). Le développement sera réalisé de manière **collaborative** par **tous les étudiants** du groupe.

L'utilisation d'un **repository GIT BitBucket** est obligatoire (il sera créé et rendu accessible par le chargé de cours). Son utilisation **correcte** et **régulière** (au minimum un commit par session de travail, libellés clairs pour les commits, permettant de suivre l'historique du projet, ...) par tous les membres de l'équipe fera partie des critères d'évaluation. L'utilisation de branches est laissée libre et à l'appréciation de chaque étudiant (sauf la branche `master` / `main` qui est obligatoire). Pour les messages des commits, vous devez respecter la convention de nommage décrite ci-dessous.

Si vous le souhaitez, vous pouvez ajouter des **fonctionnalités supplémentaires après accord du chargé de cours**. Celles-ci seront valorisées.

La version finale de chaque itération de votre projet fera l'objet d'un commit dans la branche `master` dans BitBucket et sera marquée par un **Tag**. Elle sera constituée des éléments obligatoires suivants :

- le **code source** de l'application, sous forme d'un projet respectant la structure de fichiers du framework vue au cours et compatible avec l'environnement de développement Visual Studio Code ;
- tous les fichiers externes utilisés par votre application ;
- dans le fichier `README.md` de votre projet, une **note de livraison** destinée à fournir au chargé de cours des renseignements que vous jugerez utile de porter à son attention. Au minimum vous y indiquerez les noms des utilisateurs et leur mot de passe, les bugs connus, les fonctionnalités pas implémentées ou implémentées partiellement et les fonctionnalités supplémentaires.
- la **grille d'auto-évaluation** ; cette grille et les consignes correspondantes vous seront fournies dans le courant de l'itération 1.

Afin de rendre l'installation de votre application sur une autre machine la plus simple possible, votre dépôt BitBucket sera initialisé avec le code source de base qui respecte les règles suivantes (vous veillerez à ne pas les enfreindre pendant vos développements) :

- L'URL de base de votre application sera `http://localhost/prwb_2526_xyy/`, où `x` représente la lettre de votre classe (`a`, `c`, `d`, `f` ou `g`) et `yy` représente le numéro de

votre projet exprimé sur deux chiffres (ex: classe `A`, projet `2` => `http://localhost/prwb_2526_a02/`).

- Le nom de votre projet sera `prwb_2526_xyy`.
- Vos scripts SQL pour la création et l'initialisation de la base de données se trouvera dans le sous-répertoire `database` et porteront les noms `prwb_2526_xyy.sql` (schéma) et `prwb_2526_xyy_dump.sql` (données). La base de données en elle-même portera le nom `prwb_2526_xyy` et vous y accéderez avec l'utilisateur `root` et un mot de passe `root`.
- Les fichiers `prwb_2526_xyy.sql`, `prwb_2526_xyy_dump.sql`, `dev.ini`, `prod.ini` et `.htaccess` seront adaptés en conséquence.

Démonstration pendant les cours

Très régulièrement, chaque groupe devra présenter l'état d'avancement de son projet. Ces démonstrations permettront au chargé de cours de vérifier que le projet avance correctement et de donner des conseils ou des orientations si nécessaire. La présence de tous les membres de l'équipe est obligatoire à chaque cours.

Déploiement sur un serveur

A partir de la seconde itération, vous devrez obligatoirement déployer votre application sur un serveur de test. Chaque étudiant recevra un accès à une machine virtuelle sur laquelle il devra déployer **individuellement** l'application. Ce déploiement est facultatif mais conseillé pour la première itération.

Les URL's des versions déployées par chacun des membres du groupe seront renseignées dans le fichier `README.md`.

Vous aurez bientôt accès à un tutoriel dans lequel vous déploierez le projet MSN. Vous pourrez vous en inspirer pour deployer le projet du cours. Nous vous conseillons vivement de le réaliser assez tôt dans votre processus de développement (même pendant l'itération 1) afin de vous familiariser avec la procédure et d'éviter les surprises / problèmes de dernière minute juste avant l'échéance de livraison.

Convention de nommage des commits

Vous devez respecter la convention de nommage des commits décrite ci-dessous. Elle est inspirée de la convention de nommage utilisée pour le projet Angular.

Les messages de vos commits doivent respecter la structure suivante :

```
<type>(<scope>): <short summary>
  |       |             |
  |       |             └⫸ Résumé de ce que vous avez mis dans le commit.
  |       |
  |       └⫸ Commit Scope: le nom de la fonctionnalité concernée par ce
commit | rien.
  |
  └⫸ Commit Type: build|docs|feat|fix|perf|refactor
```

`<type>` doit prendre une des valeurs suivantes :

- `build` : changements qui affectent les options du projet ou les dépendances externes (librairies)
- `docs` : changements qui concernent la documentation (analyse, readme, ...)
- `feat` : développement d'une nouvelle fonctionnalité
- `fix` : correction d'un bug
- `perf` : modification qui améliorer la performance
- `refactor` : modification qui se contente de restructurer le code sans corriger un bug ni apporter une nouvelle fonctionnalité

`<scope>` doit être utilisé dans le cas où les changements concernent une fonctionnalité principale (avec d'éventuels impacts sur d'autres fonctionnalités). Si le changement n'est pas lié à une fonctionnalité, mais est plus général, le scope (et les parenthèses) peut être omis.

Groupes de projet

Les projets sont dimensionnés pour être réalisables seul. Nous demandons malgré tout de les réaliser en équipes de deux ou trois pour favoriser l'apprentissage du travail de groupe et des outils collaboratifs.

Une fois les groupes de projet constitués, aucun changement n’est possible. Si un étudiant d’un groupe abandonne ou ne participe pas activement au projet, son (ou ses) binôme(s) le signalera(ont) au chargé de cours le plus rapidement possible.

Nous ne considérons pas qu'il y a moins de travail à faire quand on est à deux plutôt que seul (ou à trois plutôt qu'à deux). On peut en effet se répartir le travail, mais il y a une surcharge importante liée à la coordination (cfr votre expérience en TGPR), la compréhension et les tests de ce qu'a fait l'autre (les autres), des discussions, des avis divergents, de l’harmonisation du code, etc.

Le fait de se retrouver seul au lieu de deux (ou à deux au lieu de trois), en cours de projet, n'est donc pas un argument recevable pour justifier de n'être pas arrivé au bout du projet.

> **Important**
>
> Dans le cadre de notre projet, **tous les membres du groupe s'engagent à travailler en solidarité et à assumer collectivement la responsabilité de livrer les fonctionnalités demandées**. Chaque membre a un rôle essentiel à jouer, et la réussite du projet repose sur la contribution active de chacun.
>
> Il est important de préciser que, même si un membre venait à abandonner ou à fournir un travail inférieur aux attentes, le groupe est tenu de respecter le périmètre initial du projet et de livrer les résultats attendus. Dans ce cas, la personne concernée se verra attribuer une note inférieure, voire une note d'échec, en raison de son manque d'engagement.

Évaluation

Chaque itération sera évaluée et interviendra dans la note finale du projet. Nous vous demandons donc un travail régulier.

Le travail doit être réparti de manière **équitable sur l'ensemble des membres de l'équipe de projet**. Ceci sera vérifié, notamment, sur base de :

- la présence et la participation active pendant les cours et les démonstrations ;
- la répartition des commits sur les différents utilisateurs (nombre de commits et leur répartition dans le temps) ; il est donc fondamental que vos commits vous soient attribués et pour cela n'oubliez pas de définir votre nom et votre email dans Git. Si vous travaillez en appliquant le *pair programming*[1], arrangez-vous pour que la répartition des commits entre vous soit la plus homogène possible.

L'évaluation du projet prendra également en compte le sérieux et l'honnêteté avec lesquels vous aurez rempli la grille d'auto-évaluation, qui vous sera communiquée en cours d'itération.

L'évaluation finale du projet aura lieu fin mai / début juin. Une seconde session aura lieu fin juin.

Outre l'évaluation du projet chaque membre du groupe sera **interrogé individuellement** lors de l'examen pour valider sa maîtrise des technologies et de l'architecture utilisées dans le cadre du projet. Cet examen permettra de confirmer ou d'invalider la note de projet pour l'étudiant.

Si le projet présenté en première session est jugé trop peu avancé ou trop insuffisant, il n'y aura pas de défense individuelle et les étudiants du groupe seront ajournés en première session.

Si le chargé de cours estime qu'il est possible pour les étudiants de finaliser leur projet dans l'intervalle de temps entre les deux sessions, il le leur signalera. Dans le cas contraire, les étudiants n'auront pas accès à la seconde session.

La note individuelle de chaque étudiant sera fonction de la qualité du projet remis, de sa participation active et régulière et de la qualité de sa défense individuelle.

Si la participation d'un étudiant au projet pendant l'année est jugée insuffisante ou trop peu régulière, l'étudiant sera ajourné en première session et se verra refuser l'accès à la seconde session.

Fraude

Ce travail doit être réalisé en groupes de projet. Chaque groupe produira une réalisation originale qui lui est propre. Toute tentative de réutilisation, même partielle, du travail d'un autre groupe (de la même classe ou d'une autre classe) ou d'une autre personne sera considérée comme de la fraude.

En cas de fraude avérée, les projets concernés recevront une note nulle. Veuillez noter que la fraude et la récidive de fraude peuvent également faire l'objet de sanctions disciplinaires (voir le ROI du Bachelier en Informatique).

Planning

L'itération 1 de votre projet devra être livrée au plus tard le dimanche premier février 2026 à 23h59.

Bon travail !

---

1. Le *Pair programming* est une méthode de travail dans laquelle deux développeurs travaillent ensemble sur un même poste de travail. La personne qui rédige le code est appelée conducteur (*driver*). La seconde personne, appelée observateur (*observer*), assiste le conducteur en décelant les imperfections, en vérifiant que le code implémente correctement le design et en suggérant des alternatives de développement. Les rôles s'échangent régulièrement pendant la séance de programmation (Wikipedia). ↩